Problem A. the number of SYSU

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

The annual Sun Yat-sen University Programming Contest has begun again. To celebrate this event, you need to count the number of occurrences of `SYSU`.

You are given a grid with n rows and m columns. Each cell in the grid contains an uppercase letter. Your task is to determine how many times `SYSU` appears in the grid, including both horizontal occurrences (from left to right) and vertical occurrences (from top to bottom). Note that different occurrences of `SYSU` may share the same grid cells.

Input

The first line contains two integers n, m ($1 \leq n, m \leq 1000$).

The next n lines each contain m uppercase letters. The j-th character in the i-th line represents the letter at the i-th row(from top to bottom) and j-th column(from left to right) of the grid.

Output

An integer, which is the number of occurrences of `SYSU`.

Examples

standard input	standard output
5 5 SUSYS UYUUS SYSYS YUUUS SSSYY	0
4 4 ASZJ SYSU SSAA SUPJ	2

Problem B. candy

Input file:	`standard input`
Output file:	`standard output`
Time limit:	0.5 seconds
Memory limit:	256 megabytes

Dr.Z loves eating candy. One day, Dr.Z received n candies arranged in a row. Each candy has a value a_i representing its sweetness. Little Z can only eat a contiguous segment of candies, and to avoid cavities, the total sweetness of candies he eats must not exceed m. Dr.Z wants to know the maximum number of candies he can eat.

Input

The first line contains two integers n, m ($1 \leq n \leq 200000, 0 \leq m \leq 10^9$), representing the number of candies and the maximum total sweetness.

The second line contains n integers a_i ($0 \leq a_i \leq 10^9$), representing the sweetness of the i-th candy.

Output

An integer,the maximum number of candies Dr.Z can eat.

Example

standard input	standard output
5 5 0 2 3 2 2	3

Problem C. triangle

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Dr.Z loves triangles. One day, he obtained a sequence l_n of length n, and he wants to know how many disordered triplets i, j, k (satisfying $1 \leq i < j < k \leq n$) exist such that $l_i + l_j > 0$, $l_j + l_k > 0$, $l_i + l_k > 0$, and there exists a triangle with side lengths $l_i + l_j$, $l_j + l_k$, and $l_i + l_k$.

Input

The first line contains an integer n ($3 \leq n \leq 10^6$), representing the length of the sequence.

The second line contains n integers l_i ($-10^9 \leq l_i \leq 10^9$), representing the length of the sequence.

Output

An integer representing the number of triplets that meet the requirements.

Examples

standard input	standard output
5 -2 4 4 -1 0	0
4 1 3 5 -10	1

Problem D. Exchange Courses

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

As the new semester begins, it's that time again for course selection. Student A got into Course A but not into Course B. Meanwhile, some other students are also unsatisfied with the courses they have selected. So, they came up with an idea: they could exchange courses with each other.

Specifically, suppose Student A and Student B have selected Course A and Course B respectively; after exchanging, the courses selected by Student A and Student B become B and A respectively. Except for Student A, the other students only want to exchange for their desired course and are unwilling to accept any other course. Student A, however, does not require getting the desired course in the first exchange.

Student A wants to know whether it is possible to get the desired course through exchanges, and if so, what is the minimum number of exchanges required? There are n other students who want to exchange courses besides Student A.

Input

The first line contains a positive integer n ($1 \le n \le 10^5$), indicating the number of students who want to exchange courses besides student A.

The next n lines each contain two strings, s_i and t_i, separated by a single space. This indicates that the i-th student was assigned course s_i but wants to exchange to course t_i.

The last line contains two strings, a and b, indicating that student A was assigned course a and wants to exchange to course b.

Additional constraint on the input: $|a|, |b|, \sum(|s_i| + |t_i|) \le 10^6, \forall i, s_i \ne t_i, a \ne b$,It is guaranteed that all courses' names consist only of uppercase letters, lowercase letters, and digits, and do not contain any other special characters.

Output

Output a single line indicating the minimum number of exchanges required. If it is impossible to obtain the desired course, output -1.

Examples

standard input	standard output
3 math English Chinese math Biology Chinese math Biology	2
3 math English Chinese math Biology Chinese Biology math	-1

Note

In the first example, Student A can first exchange courses with the second student, after which Student A gets Chinese. Then, Student A can exchange courses with the third student, after which Student A gets

Biology, thus obtaining his desired course. A total of two exchanges were made, and it can be proven that this is the minimum number of exchanges.

Problem E. Test

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Student A took an exam. The exam consists of n multiple-choice questions. Each multiple-choice question has m options, numbered from 1 to m. Student A's answer sheet can be represented by an array a, where a_i is the option number chosen by Student A for the i-th question. Student A's answer sheet must satisfy the following condition: $\forall 1 \leq i < n, a_i \leq a_{i+1}$. Student A will ultimately select one of all possible answer sheets that satisfy the condition uniformly at random for submission.

Given the correct answers b_i for all n multiple-choice questions, find the sum of the number of questions answered correctly across all possible answer sheets Student A could have submitted. Please output the result modulo 998244353.

You may use the following:The modular multiplicative inverse of an integer a modulo a prime modulus M (like 998244353) is an integer x such that $a \cdot x \equiv 1 \pmod{M}$. It exists when $\gcd(a, M) = 1$. This allows for modular division, defined as $\frac{b}{a} \equiv b \cdot x \pmod{M}$. One method to compute the inverse is $x \equiv a^{M-2} \pmod{M}$.

Input

The first line contains two positive integers, n ($1 \leq n \leq 10^5$) and m ($1 \leq m \leq 10^5$).

The second line contains n positive integers, where the i-th integer is b_i ($1 \leq b_i \leq m$), representing the answer to the i-th question.

Output

A non-negative integer representing the result of the sum of the number of correctly answered questions across all possible answer sheets by Student A, taken modulo 998244353.

Example

standard input	standard output
2 3 1 3	6

Note

We use an ordered pair (t_1, t_2) to represent Student A's answer sheet, where t_1 is the option chosen for the first question and t_2 for the second question. Student A has six possible answer sheets: $(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)$. The numbers of correctly answered questions for each are $1, 1, 2, 0, 1, 1$, respectively. Thus, the answer is $1 + 1 + 2 + 0 + 1 + 1 = 6$.

Problem F. Dr.Z, the Raider

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	256 megabytes

`Dr.Z` is preparing to launch a major raid on the villages in `Minecraft`. This time, `Dr.Z` has set his sights on n villages. These villages are connected by $(n-1)$ roads, and each village can reach any other village, while `Dr.Z` will only move along these roads. Among these $(n-1)$ roads, the i-th $(1 \le i \le (n-1))$ road connects the u_i-th village and the v_i-th village, and has a weight w_i.

To fully collect the resources in the villages, `Dr.Z` decides to visit each village exactly once. To do so, `Dr.Z` will first select a permutation $p_1, p_2, \cdots, p_n$ from all permutations of $1, 2, \cdots, n$, and carry out the raid as follows: first, collect resources from the p_1-th village; next, starting from the p_1-th village, move to the p_2-th village in the way that minimizes the number of roads traversed, and collect resources from the p_2-th village; then, starting from the p_2-th village, move to the p_3-th village in the way that minimizes the number of roads traversed, and collect resources from the p_3-th village; ...; finally, starting from the p_{n-1}-th village, move to the p_n-th village in the way that minimizes the number of roads traversed, and collect resources from the p_n-th village.

Since `Dr.Z` is already a legendary figure in this world, `Dr.Z` can make extensive use of ender pearls, causing the cost $\text{cost}(x, y)$ for `Dr.Z` to move from the x-th village to the y-th village in the way that minimizes the number of roads traversed to become the `bitwise XOR` value of the weights of all roads that `Dr.Z` would pass through during this process. That is, suppose `Dr.Z` needs to pass through roads $e_1, e_2, \cdots, e_m$ during this process, then $\text{cost}(x, y) = \oplus_{i=1}^{m} w_{e_i}$, where $\oplus$ denotes the `bitwise XOR` operation. Meanwhile, `Dr.Z` considers the total cost of the raid, $f(p_1, p_2, \cdots, p_n)$, as the sum of the costs of the $(n-1)$ moves made by `Dr.Z` during the raid. That is, $f(p_1, p_2, \cdots, p_n) = \sum_{i=1}^{n-1} \text{cost}(p_i, p_{i+1})$.

`Dr.Z` does not care about the total cost of the raid. What `Dr.Z` cares about is, for all possible permutations $p_1, p_2, \cdots, p_n$ of $1, 2, \cdots, n$, what is the sum of the total costs of the raids carried out according to the permutations, or, the sum of $f(p_1, p_2, \cdots, p_n)$ in short. Since this number may be very large, you only need to tell `Dr.Z` its remainder when divided by 998244353.

Input

The first line contains a positive integer n $(2 \le n \le 2 \times 10^5)$.

The next $(n-1)$ lines each contain three non-negative integers. In these $(n-1)$ lines, the three numbers in the i-th line $(1 \le i \le n-1)$ represent u_i, v_i, and w_i respectively $(1 \le u_i, v_i \le n, 0 \le w_i < 2^{30})$.

Output

A single integer, the answer to the problem.

Example

standard input	standard output
5 1 2 1 1 3 2 2 4 3 2 5 4	1632

Note

Suppose that $p_1 = 1$, $p_2 = 2$, $p_3 = 3$, $p_4 = 4$, and $p_5 = 5$.

Then,

$$\begin{aligned}
&\text{cost}(p_1, p_2) = \text{cost}(1, 2) = w_1 = 1 \\
&\text{cost}(p_2, p_3) = \text{cost}(2, 3) = w_1 \oplus w_2 = 1 \oplus 2 = 3 \\
&\text{cost}(p_3, p_4) = \text{cost}(3, 4) = w_2 \oplus w_1 \oplus w_3 = 2 \oplus 1 \oplus 3 = 0 \\
&\text{cost}(p_4, p_5) = \text{cost}(4, 5) = w_3 \oplus w_4 = 3 \oplus 4 = 7
\end{aligned}$$

Therefore,

$$\begin{aligned}
& f(p_1, p_2, p_3, p_4, p_5) \\
= \ & \text{cost}(p_1, p_2) + \text{cost}(p_2, p_3) + \text{cost}(p_3, p_4) + \text{cost}(p_4, p_5) \\
= \ & 1 + 3 + 0 + 7 \\
= \ & 11
\end{aligned}$$

Please remind that `bitwise XOR` is a binary operation that compares each bit of two numbers and returns `1` if the bits are different, and `0` if they are the same.

Problem G. Dr.Z's Trial

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

`Dr.Z` is participating in a trial. `Dr.Z` has a set of numbers in front of him, initially containing n non-negative integers $v_1, v_2, \cdots, v_n$. `Dr.Z` can choose a non-negative integer m, and then engage in m rounds of confrontation with the trial system. In each round of confrontation:

1. `Dr.Z` selects two different numbers a and b from the set, and chooses a number c from $\{0, 1\}$;
2. The trial system calculates d based on `Dr.Z`'s choice. Specifically, if $c = 0$, then $d = a\&b$; if $c = 1$, then $d = a \oplus b$. Here, & represents the `bitwise AND` operation, and $\oplus$ represents the `bitwise XOR` operation;
3. If d is not among the numbers in front of `Dr.Z`, then d is added to the numbers in front of `Dr.Z`.

After all m rounds of confrontation are complete, the maximum value among the numbers in front of `Dr.Z` will become `Dr.Z`'s score. `Dr.Z` hopes to achieve the best possible score while also wanting to finish the trial as quickly as possible. `Dr.Z` wants you to determine the maximum possible score s for `Dr.Z`, and at the same time, provide a strategy that involves no more than 3000 confrontations while ensuring that `Dr.Z`'s score is s. Naturally, things don't always have ideal outcomes. If `Dr.Z`'s score can be infinitely large, or if `Dr.Z` cannot achieve a score of s with no more than 3000 confrontations, you need to explain to him that his wishes cannot be fulfilled.

Input

The first line of input contains a non-negative integer, representing T ($1 \le T \le 10^3$), the number of test cases. Then, for each test case:

- The first line of input contains a non-negative integer, representing n ($1 \le n \le 5 \times 10^5$).
- The second line of input contains n non-negative integers, separated by a single space between adjacent numbers. These numbers represent $v_1, v_2, \cdots, v_n$ ($\forall i = 1, 2, \cdots, n, 0 \le v_i < 2^{30}$); for any integers i and j satisfying $1 \le i < j \le n$, $(v_i \ne v_j)$.

It is guaranteed that the sum of n in all test cases does not exceed 5×10^5.

Output

For each test case:

- If `Dr.Z`'s score can be infinitely large, the program should output a single line containing `infinite score` and go to the next test case. Otherwise, the program should first output a non-negative integer s on a single line.
- Next, if `Dr.Z` cannot achieve a score of s with no more than 3000 confrontations, the program should output an additional line containing `no way to end early` and go to the next test case. Otherwise, the program should output a strategy that satisfies `Dr.Z`'s requirements.
- To output a strategy that satisfies `Dr.Z`'s requirements, the program should output a non-negative integer m ($0 \le m \le 3000$) on a single line. Then, in the next m lines, each line should output three non-negative integers separated by a single space. The three non-negative integers in the i-th line ($1 \le i \le m$) represent a_i, b_i, c_i ($a_i \ne b_i$, $c_i \in \{0, 1\}$). The program must ensure that in the i-th confrontation ($1 \le i \le m$), both a_i and b_i are present in the set of numbers that `Dr.Z` has before the i-th confrontation.

Example

standard input	standard output
2 4 3 1 2 4 3 2 0 1	7 4 1 2 1 3 4 0 4 3 1 2 0 1 3 1 1 2 1

Note

In the first test case, `Dr.Z` can do the following to achieve the maximum possible score 7:

1. Choose $a_1 = 1$, $b_1 = 2$ and $c_1 = 1$. This will make $d = a_1 \oplus b_1 = 3$. Since 3 has been in the numbers in front of `Dr.Z`, d will not be added into the numbers;
2. Choose $a_2 = 3$, $b_2 = 4$ and $c_2 = 0$. This will make $d = a_2 \& b_2 = 0$. Since 0 is not in the numbers in front of `Dr.Z`, 0 is then added into these numbers;
3. Choose $a_3 = 4$, $b_3 = 3$ and $c_3 = 1$. This will make $d = a_3 \oplus b_3 = 7$. Since 7 is not in the numbers in front of `Dr.Z`, 7 is then added into these numbers;
4. Choose $a_4 = 2$, $b_4 = 0$ (note that 0 has been added into the numbers in front of `Dr.Z` in the second step) and $c_3 = 1$. This will make $d = a_3 \oplus b_3 = 2$. Since 2 has been in the numbers in front of `Dr.Z`, d will not be added into the numbers.

After the above operations, the numbers in front of `Dr.Z` are $0, 1, 2, 3, 7$ in ascending order. So the maximum value among these numbers is 7, and thus `Dr.Z`'s score is 7. It can be shown that `Dr.Z` cannot get a score higher than 7.

Please remind that `Bitwise AND` is a binary operation that compares each bit of two numbers and returns `1` only if both bits are `1`, otherwise returns `0`. Please also remind that `bitwise XOR` is a binary operation that compares each bit of two numbers and returns `1` if the bits are different, and `0` if they are the same.